Algorithms 2

Lecture 9

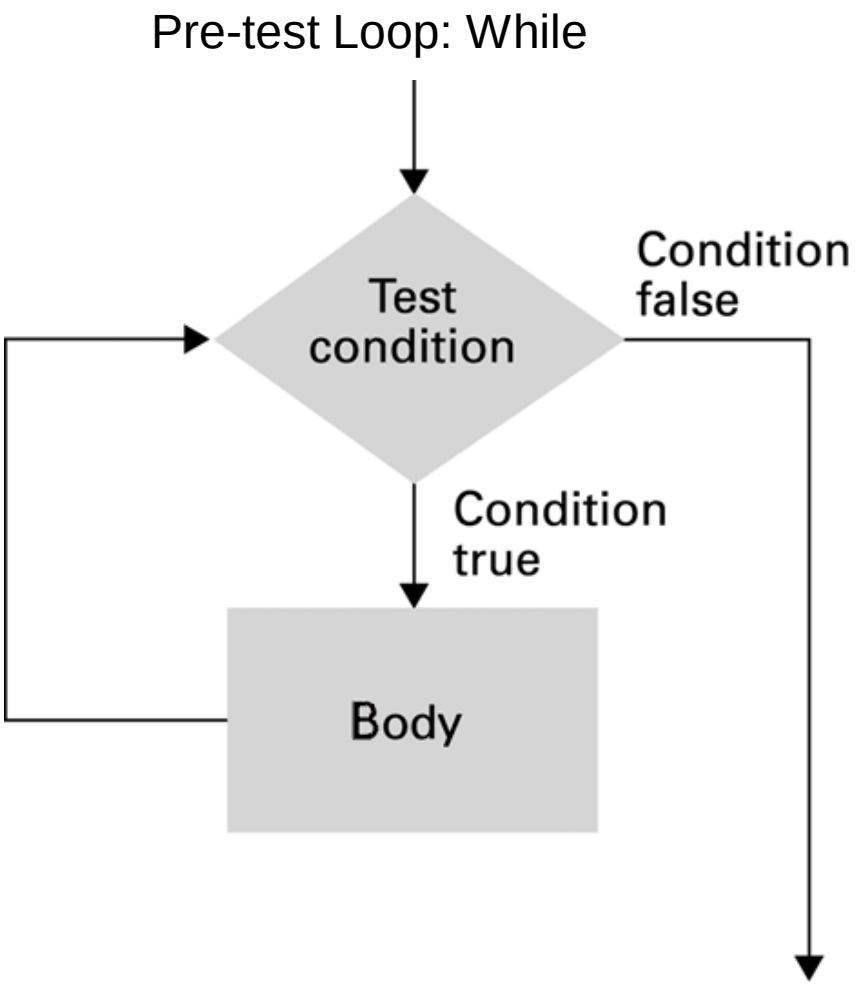
Outline

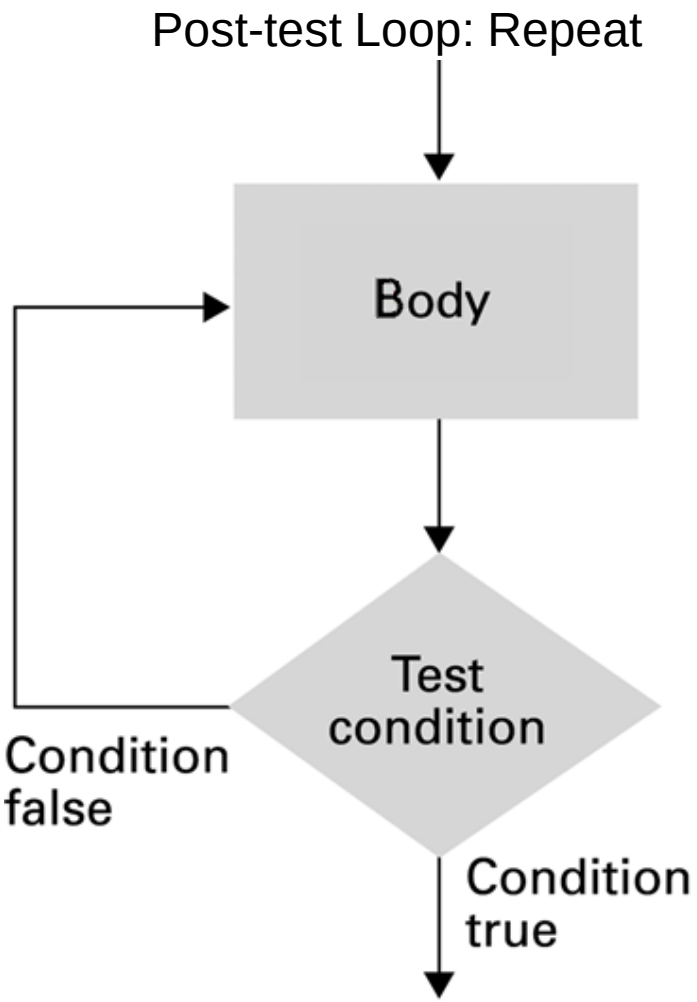
- Iterative Structures
  - Insertion sort
  - Sequential search
- Recursive Structures
  - Binary search
  - Mondrian art
  - Merge sort (optional)
- Algorithm Efficiency
- Algorithm Correctness

Iterative Structures

- It is often happens that an algorithm contains repeated actions
- Each action is similar to previous one
- These structures are implemented using loops.
- Two types of loops:
  - Pre-test loop
  - Post-test loop

Loop Types


Pre-test Loop: While
Test condition
Condition false
Condition true
Body


Post-test Loop: Repeat
Body
Test condition
Condition false
Condition true

Loop Types in pseudocode

- Pretest loop:

```
while (condition):
    body
```

- Posttest loop:

```
repeat:
    body
    until(condition)
```

Components of repetitive control

Initialize: Establish an initial state that will be modified toward the termination condition

Test: Compare the current state to the termination condition and terminate the repetition if equal

Modify: Change the state in such a way that it moves toward the termination condition

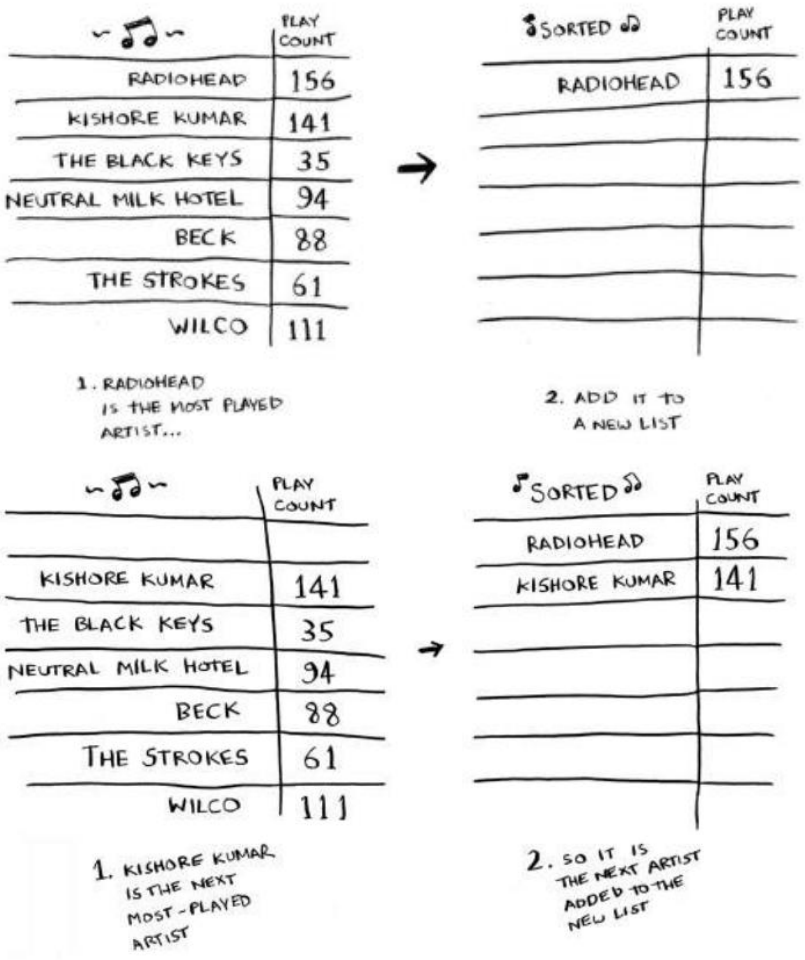
Sorting and Searching

- Sorting problem involves rearranging the elements of an array according to some order
- In Computer Science, sorting algorithms are taught to showcase a range of algorithmic ideas that are used in designing efficient and correct algorithms
- Sorting algorithms are implemented using:
  - Comparison/non-comparison based strategies
  - Iterative vs Recursive implementation
  - Divide-and-Conquer paradigm
  - Best/Worst/Average-case time/space complexity
  - Randomized algorithms

Uses of Sorting Algorithms

- Searching for a specific value v in array A
- Finding min/max/k-th smallest/greatest value in array A
- Testing for uniqueness and removing duplicates in array A
- Counting how many times a value v appear in array A
- Set intersection/union between sorted arrays A and B
- Finding a target pair x from A and y from B, such that $x+y=z$
- And many more…

Selection Sort



More Selections …

Pseudocode: Selection Sort

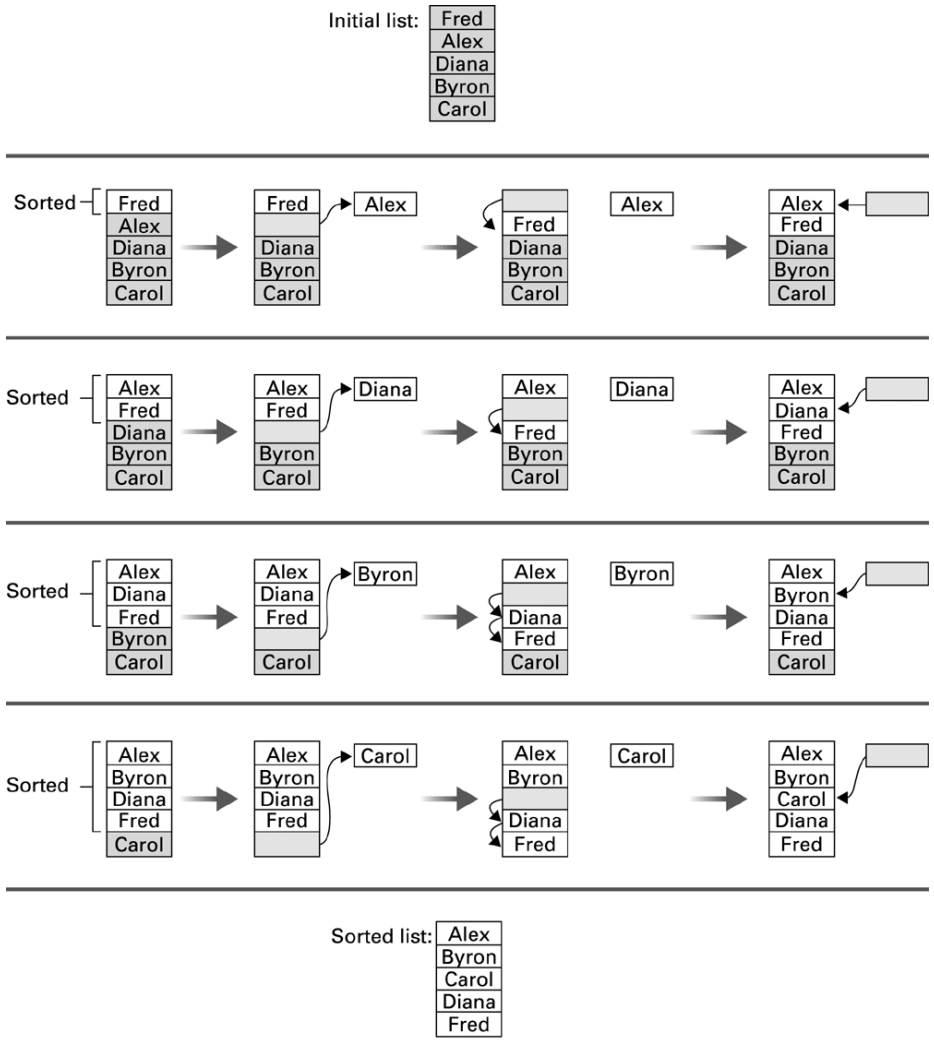
```
def sort(list):
      i = 0
       while (i <= length of List):
          minIndex = i
          j = i+1
          while (j <= length of List):
               if (list[j] < list[minIndex])
                   minIndex = j
               j+=1
          Swap list[i] and list[minIndex] in list
          i+=1
```

Python Implementation: Selection Sort

```
def selectionSort(array, size):
    for ind in range(size):
        min_index = ind

        for j in range(ind + 1, size):
            # select the minimum element in every iteration
            if array[j] < array[min_index]:
                min_index = j
        # swapping the elements to sort the array
        (array[ind], array[min_index]) = (array[min_index], array[ind])
```

Insertion Sort



Pseudocode: Insertion Sort

```
def Sort(List):
    N = 2
    while (N <= length of List):
        Pivot = List[N]
        K = (N-1) // index adjacent to Pivot
        while (K>0 and List[K] > Pivot):
            Swap List[K] and Pivot in List
            K = K - 1
        N = N + 1
```

Python Implementation: Insertion Sort

```
def insertion_sort(lst):
    n = 1
    while (n < len(lst)):
        pivot = lst[n]
        key = n - 1
        while (key >= 0 and lst[key] > pivot):
            # this is swap operation in python
            lst[key + 1], lst[key] = lst[key], pivot
            key = key - 1
        n = n + 1
    return lst
```

Pseudocode: Sequential Search

```
def Search (List, TargetValue):
    if (List is empty):
        Declare search a failure
    else:
        Select the first entry in List to be TestEntry
        while (TargetValue > TestEntry and entries remain):
            Select the next entry in List as TestEntry
        if (TargetValue == TestEntry):
            Declare search a success
        else:
            Declare search a failure
```

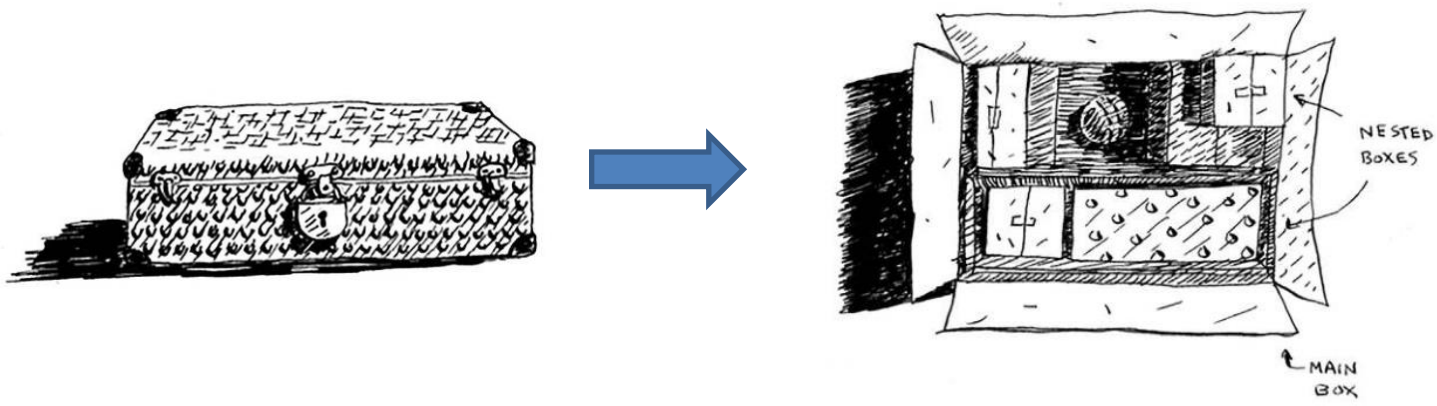
Python Implementation: Sequential Search

```
def seq_search(lst, targetValue):
    if len(lst) == 0: return False
    i = 0
    while (targetValue > lst[i] and i < len(lst)):
        i += 1

    if lst[i] == targetValue:
        return True
    else:
        return False
```

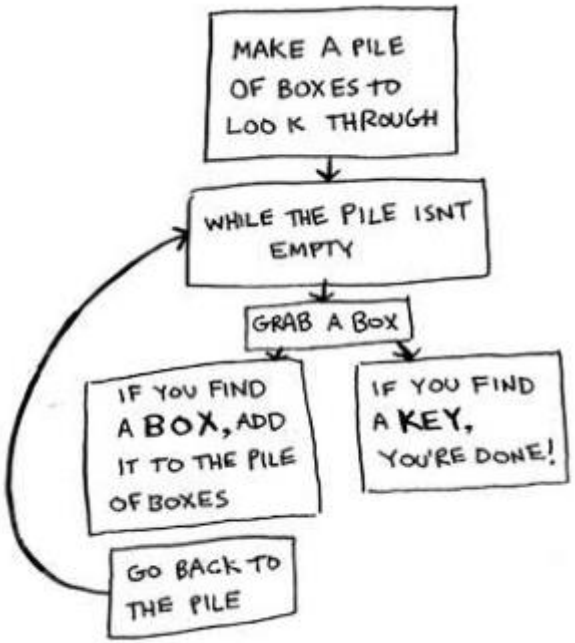
Recursion

- The execution of a procedure leads to another execution of the same procedure.
- Multiple activations of the procedure are formed, all but one of which are waiting for other activations to complete.



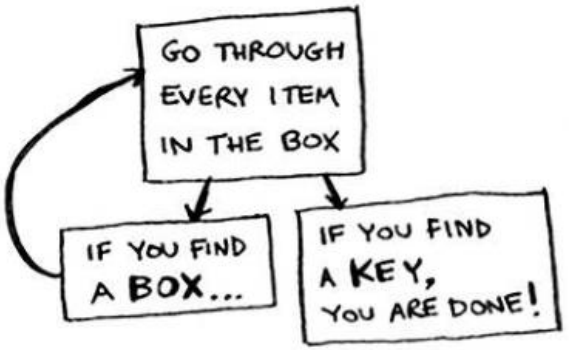
Finding a key in the box

Iterative approach




```
def look_for_key(main_box):
  pile = main_box.make_a_pile_to_look_through()
  while pile is not empty:
    box = pile.grab_a_box()
    for item in box:
      if item.is_a_box():
        pile.append(item)
      elif item.is_a_key():
        print "found the key!"
```

Recursive approach



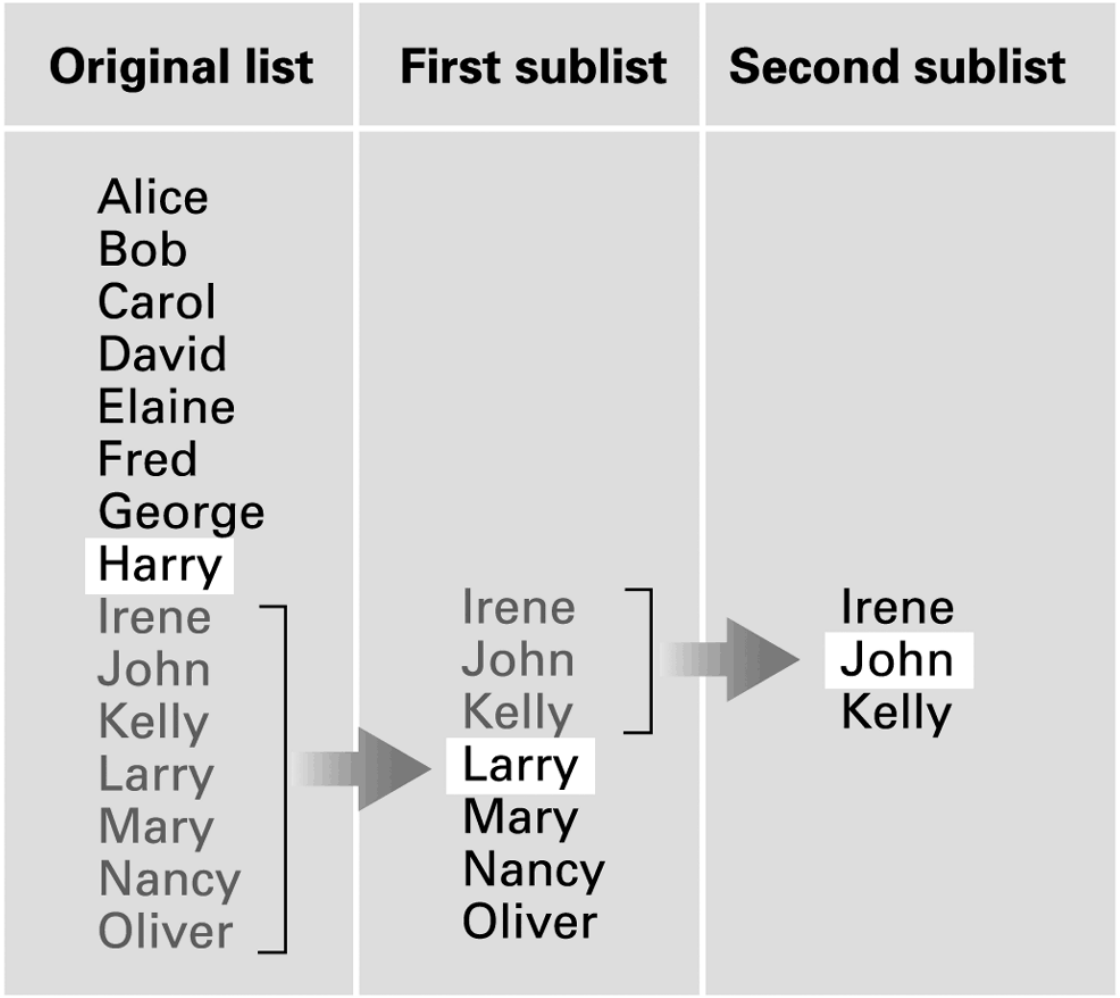

```
def look_for_key(box):
  for item in box:
    if item.is_a_box():
      look_for_key(item)
    elif item.is_a_key():
      print "found the key!"
```

Recursion!

Components of Recursive Algorithm

- **Divide –** break the problem into a number of subproblems.
- **Conquer –** reduce the problem to its smallest form where the solution is trivial.
- **Combine –** derive the solution of a bigger problem from the solutions of smaller problems.

Binary Search



Binary Search Pseudocode - Draft

```
if (List is empty):
    Report that the search failed
else:
    TestEntry = middle entry in the List
    if (TargetValue == TestEntry):
        Report that the search succeeded
    if (TargetValue < TestEntry):
        Search the portion of List preceding TestEntry for
        TargetValue, and report the result of that search
    if (TargetValue > TestEntry):
        Search the portion of List following TestEntry for
        TargetValue, and report the result of that search
```

Binary Search Pseudocode

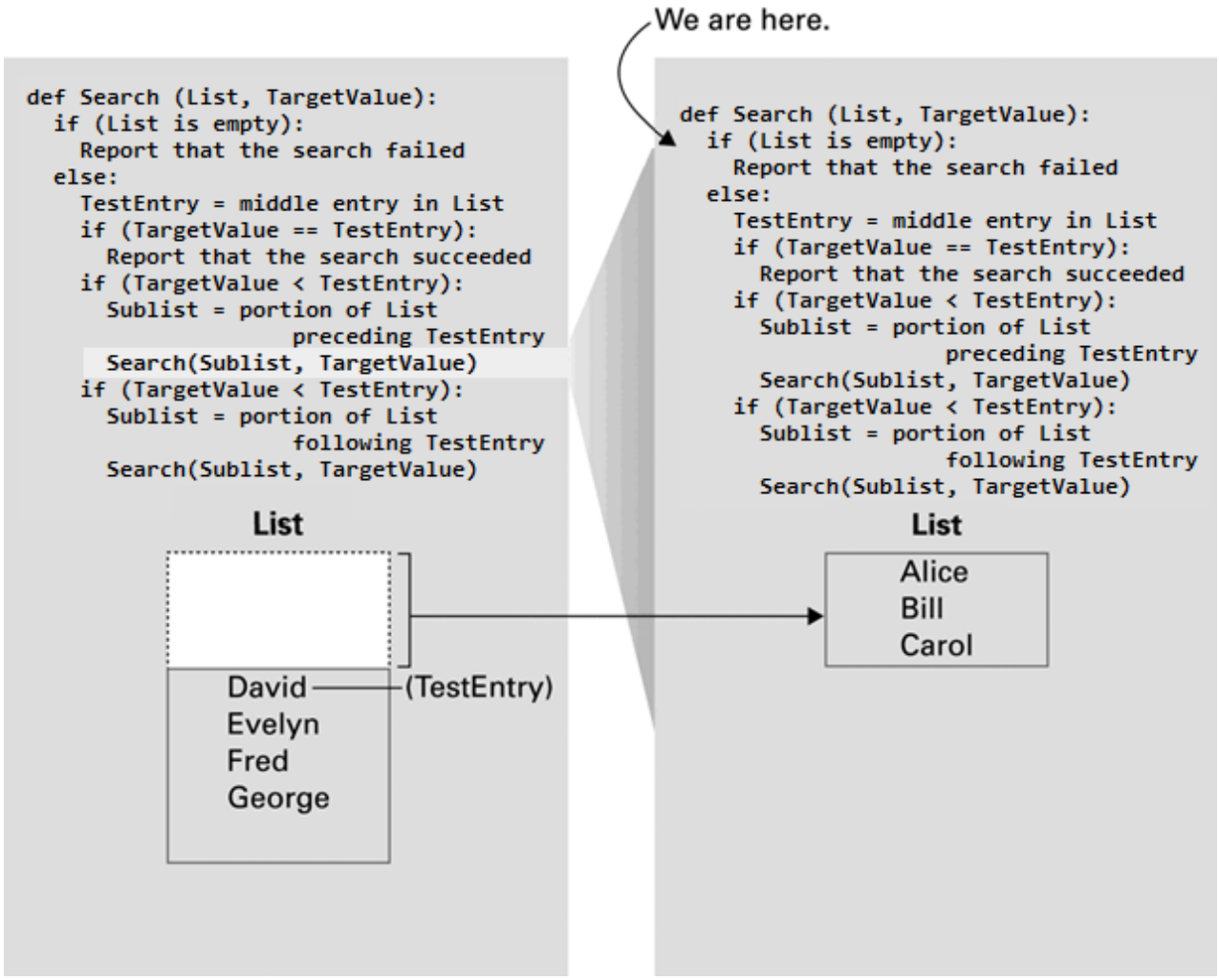
```
def Search(List, TargetValue):
    if (List is empty):
        Report that the search failed
    else:
        TestEntry = middle entry in the List
        if (TargetValue == TestEntry):
            Report that the search succeeded
        if (TargetValue < TestEntry):
            Sublist = portion of List preceding TestEntry
            Search(Sublist, TargetValue)
        if (TargetValue > TestEntry):
            Sublist = portion of List following TestEntry
            Search(Sublist, TargetValue)
```

Python Implementation: Binary Search

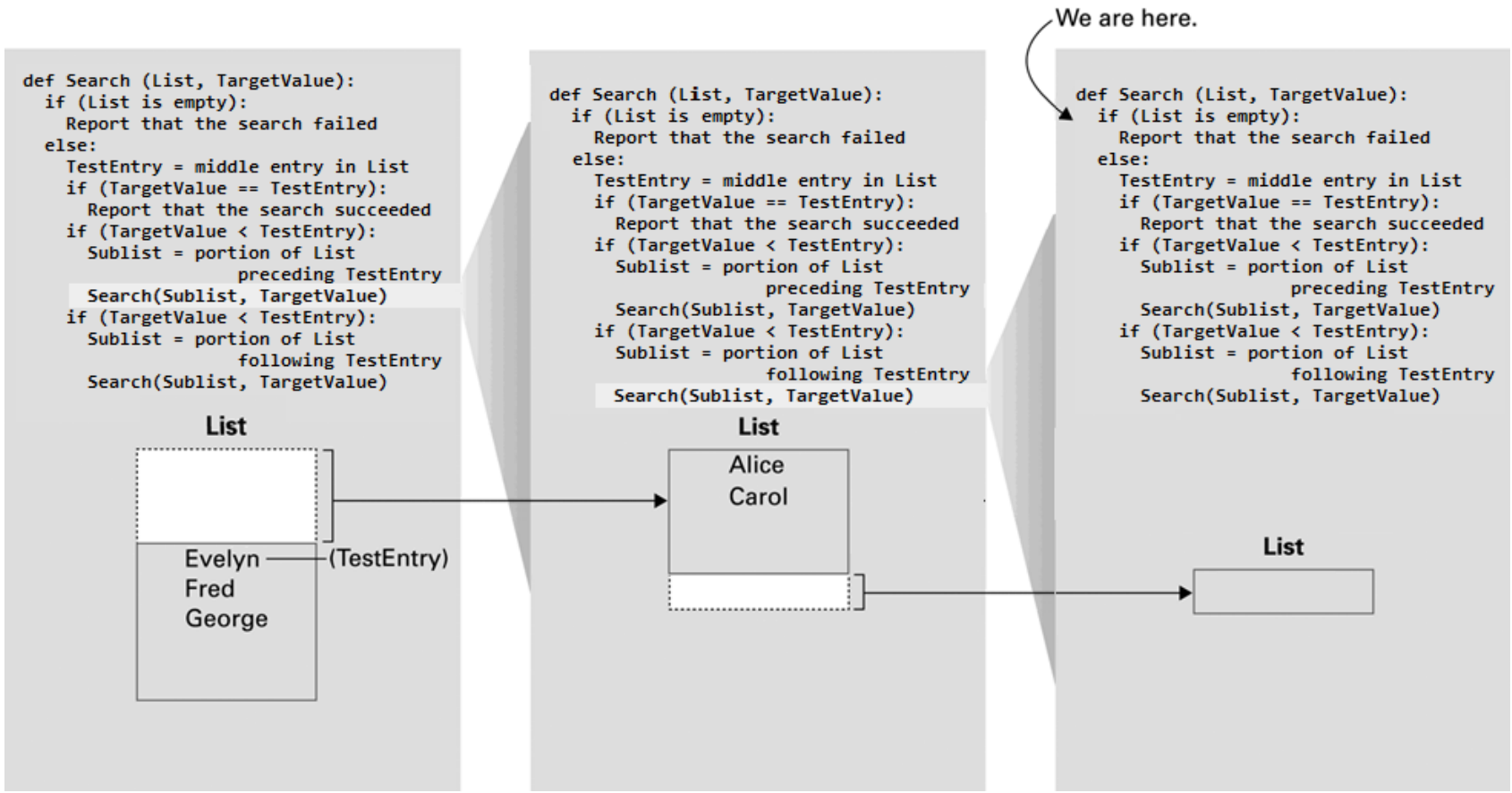
```
def bin_search(lst, targetValue):
    if len(lst) == 0: return False
    mid_idx = (len(lst) - 1) / 2
    testEntry = lst[mid_idx]
    if (targetValue == testEntry):
        return True
    if (targetValue < testEntry):
        return bin_search(lst[:mid_idx], targetValue)
    if (targetValue > testEntry):
        return bin_search(lst[mid_idx + 1:], targetValue)
```

Binary Search

- We have the list:
  - Alice
  - Bill
  - Carol
  - David
  - Evelyn
  - Fred
  - George
- Search Bill



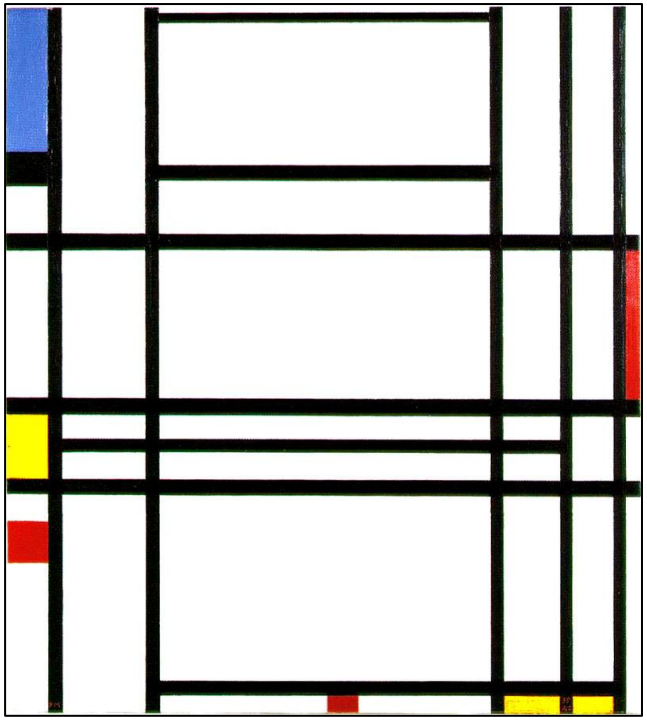
Binary Search: Search Dave



Termination Condition of Recursion

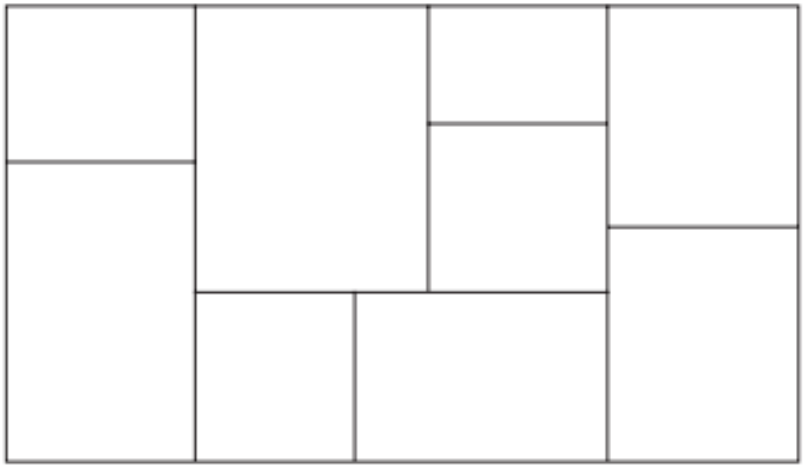
Piet Mondrian's Art

- Dutch painter lived in 1872-1944
- Founder of neoplastism



Composition No. 10

Mondrian Art using Recursion



```
def Mondrian (Rectangle):
  if (the size of Rectangle is too large for your artistic taste):
     divide Rectangle into two smaller rectangles.
     apply the function Mondrian to one of the smaller rectangles.
     apply the function Mondrian to the other smaller rectangle.
```

Algorithm Efficiency

- Measured as number of instructions executed
- When the number of input data grows linearly, what is the growth of the number of operations on an algorithm?
- The number of operations for processing a set of data depends on both the size of the data and the data pattern.
  - What is the best data pattern for insertion sort?
  - What is the worst data pattern for insertion sort?

Registrar's Search Dilemma

- Registrar has the record of 30 000 students in an ordered list.
- Computer takes 10 miliseconds to retrieve and to check particular record from this list.
- How much time on average computer would process to find a given student from the list if it uses *sequential search algorithm*? 150 seconds. Worst-case? *O(n)*
- How much time on average would it be if it uses *binary search algorithm?* 0.15 seconds. Worst case? *O(*$\log_2 n$)
- Big O notation is used to represent efficiency of algorithms
- Example:
  - Insertion sort is in $O(n^2)$,
  - Bruteforce sort *O(n!)*

Applying the insertion sort in a worst-case situation

Comparisons made for each pivot

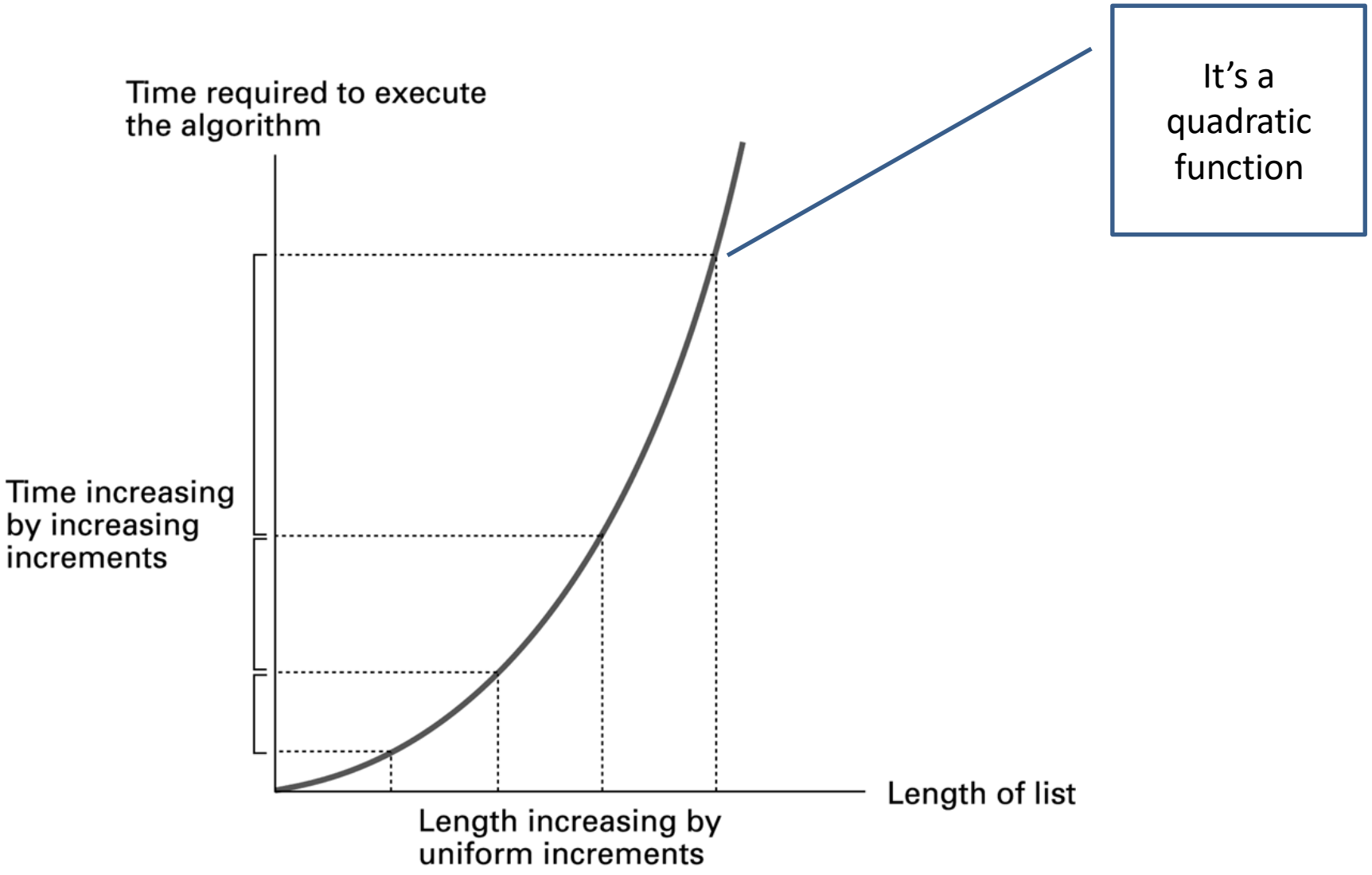
Initial list	1st pivot	2nd pivot	3rd pivot	4th pivot	Sorted list
Elaine David Carol Barbara Alfred	1 Elaine David Carol Barbara Alfred	3 David 2 Elaine Carol Barbara Alfred	6 Carol 5 David 4 Elaine Barbara Alfred	10 Barbara 9 Carol 8 David 7 Elaine Alfred	Alfred Barbara Carol David Elaine

- In best-case, the list is ordered already, only *n-1* comparisons made
- In worst-case, the list is in reverse order, so the number of comparisons arithmetically progress: 1+2+3+…+(n-1), which is equal to

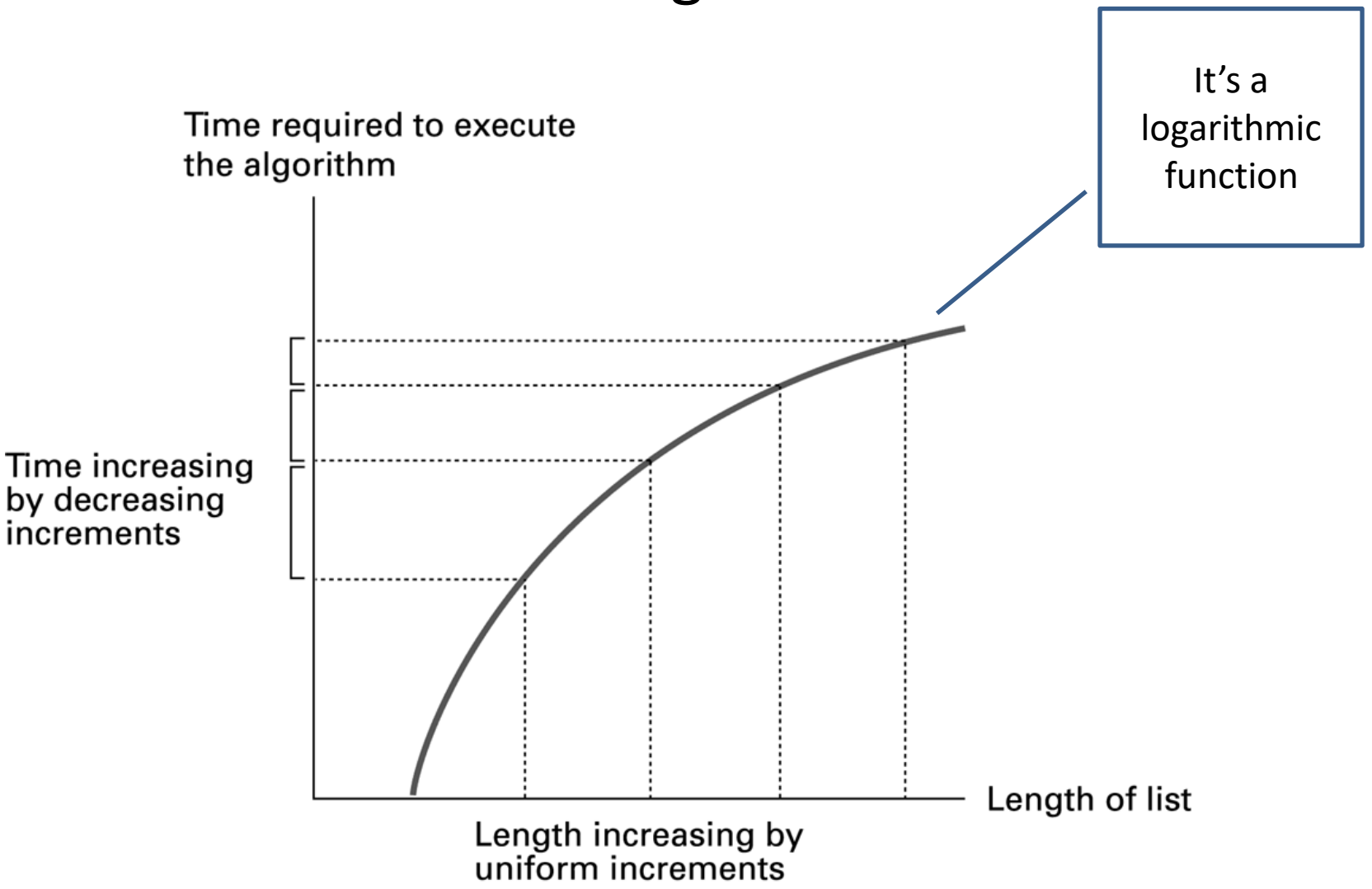
$$\frac{n(n-1)}{2} = 0.5(n^2 - n) = O(n^2)$$

- If the list contains 5 records, 10 comparisons should be made. We can see from above analysis, that the number of comparisons is in quadratic relation with the input size.

Graph of the worst-case analysis of the insertion sort algorithm



Graph of the worst-case analysis of the binary search algorithm



Algorithm Correctness

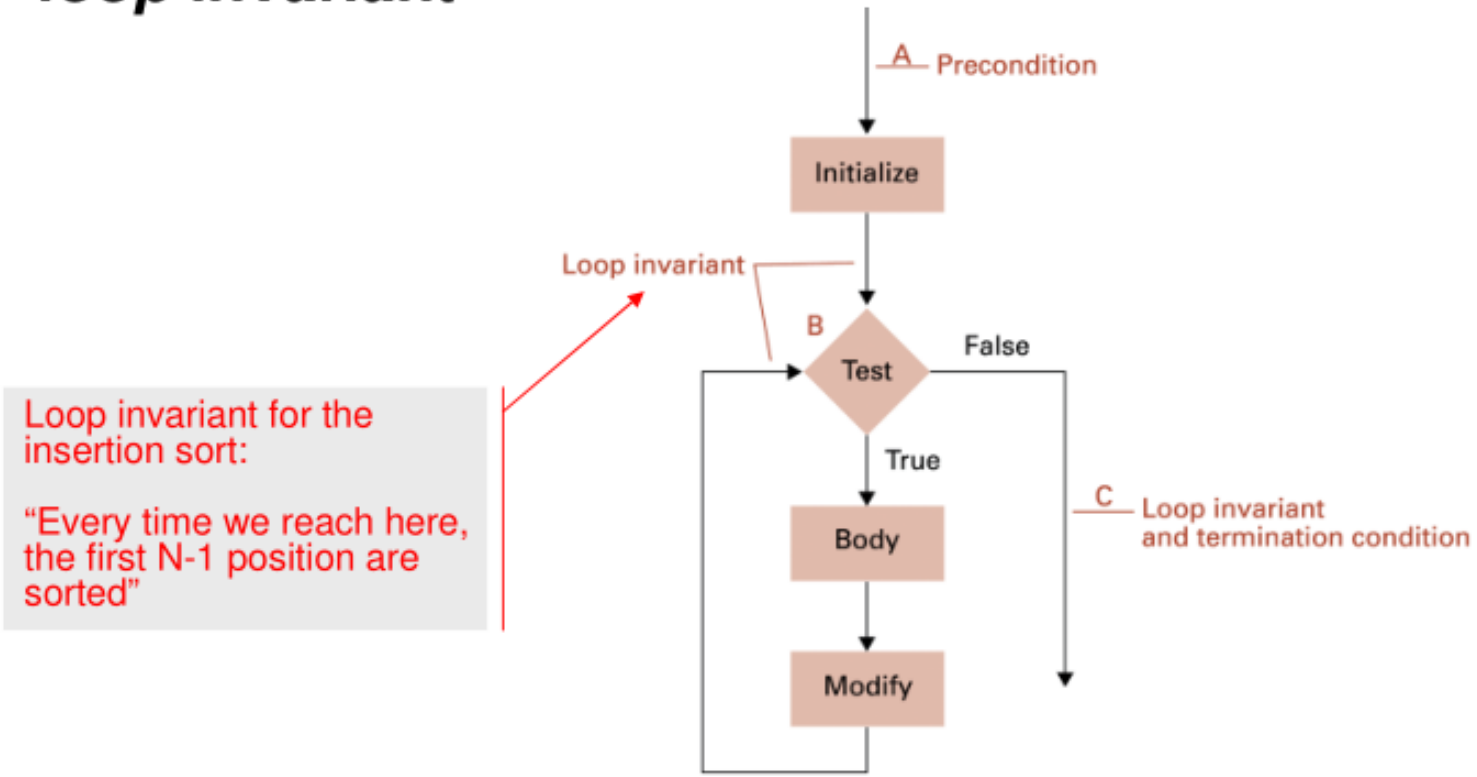
- Once we design an algorithm, the next question is how we know that it is correct?
- There are formal ways of proving the correctness of algorithms:
  - Proof by mathematical induction
  - Direct proof
  - Contrapositive Proof
  - Proof by contradiction
  - Exhaustive Proof

Algorithm Correctness: Assertions

- Assertions (claims) can be established at various points in the program
- ***Preconditions (Initialization)*** specify if the input satisfies requirements of the algorithms
  - For example, in binary search, if list is empty, search is failed
- ***Loop invariants (Maintenance)*** specify what claim about processed input can be made after each iteration.
  - For example, in insertion sort after each iteration, top N-1 lists are sorted.
- ***Postconditions (Termination)*** specify the desired characteristics of the correct output
  - For example, in insertion sort, if all elements in the list are positioned correctly, the list is sorted.

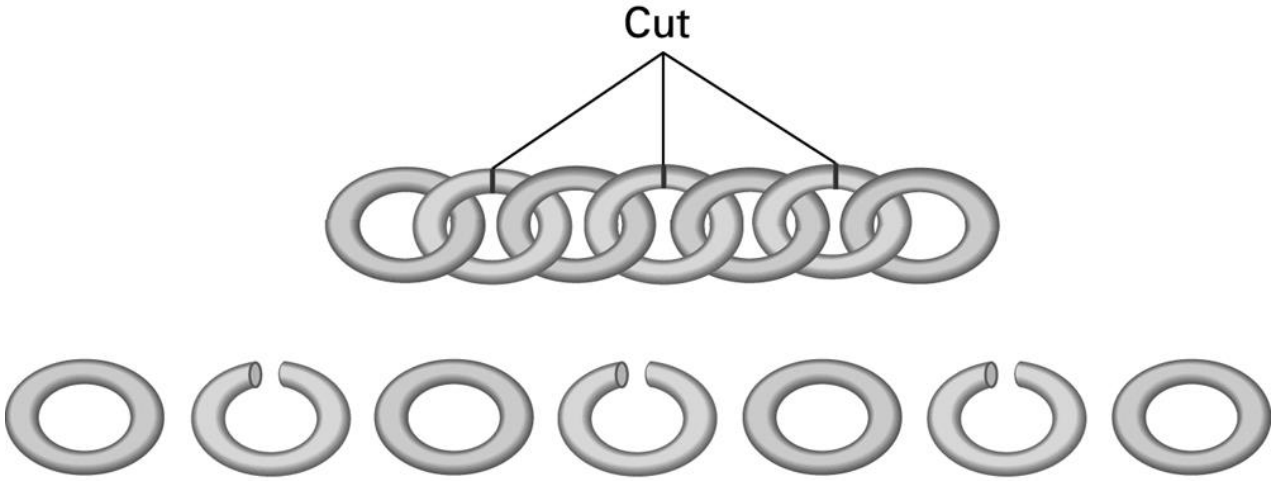
Example of Assertions

- An assertion at a point in a loop that is true every time that point in the loop is reached is known as ***loop invariant***



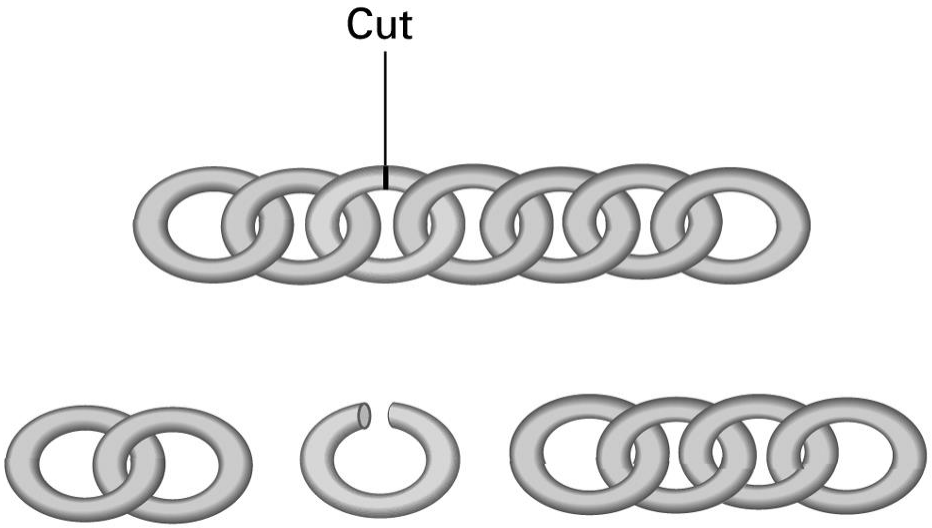
Chain Separating Problem

- Intuition is not always correct when solving problems. For example, even if algorithm works for 10 000 different inputs correctly, it does not prove that it is correct.
- A traveler has a gold chain of seven links. He must stay at an isolated hotel for seven nights. The rent each night consists of one link from the chain.
- What is the fewest number of links that must be cut so that the traveler can pay the hotel one link of the chain each morning without paying for lodging in advance?



Solving the problem with only one cut

- How it works:
  - Day 1: Give 1 ring
  - Day 2: Give 2 rings, get 1 ring back as change
  - Day 3: Give 1 ring
  - Day 4: Give 4 rings, get 3 rings back as change
  - so forth …
- This solution is correct, because less than one cut is zero cuts. We have no solution for zero cuts.



Suggested Reading/Activity

- Check the visualizations of different algorithms at visualgo.net
- Play with the visualizations of different algorithms here . Check N-Queens Problem there, review its solution.